User-Defined Functions

10 beginner Python practice tasks

Assignment requirement: Complete Task 10 (compulsory) and any six other tasks of your choice. Submit seven tasks in total.

For every task, define at least one function with `def` and call it from your program. Take the required values using `input()` and convert them with `int()` or `float()` where needed.

Use only beginner concepts: arithmetic, `if` / `elif` / `else`, nested `if`, `while`, input, and simple menu choices. Do not use lists, dictionaries, classes, or advanced libraries.

1. Two-Number Calculator

Create a function `calculate(a, b, choice)` that performs addition, subtraction, multiplication, or division based on the user's choice. Show a simple menu, read two numbers and the choice, and print the answer. Check that the user does not divide by zero.

2. Even or Odd Checker

Create a function `check_even_odd(number)` that prints whether an integer is even or odd. Ask the user for a number and call the function. **Bonus:** use a `while` loop to let the user check more numbers until they choose to stop.

3. Pass, Fail, and Grade

Create a function `show_result(mark)` that first checks whether the mark is between 0 and 100. Use `if` / `elif` to print Fail for marks below 35, and a grade for passing marks: A (90–100), B (75–89), C (60–74), or D (35–59). Call it with a mark entered by the user.

4. Simple Login Check

Create a function `login()` that asks for a username and password. Use a nested `if`: first check the username, then check the password. Print a suitable success or failure message. Choose sample credentials and display them in a comment for the person practicing.

5. Number Guessing (Three Attempts)

Choose a fixed secret number in the program. Create a function `guess_number()` that uses a `while` loop to give the user up to three attempts. After each guess, use `if` / `elif` / `else` to say whether the guess is correct, too low, or too high. Stop early if the answer is correct.

6. Basic Shopping Bill

Create a function `calculate_bill(price, quantity)` that returns or prints the total price. Ask for the item price and quantity. If the total is at least 1,000, apply a 10% discount; otherwise, do not apply a discount. Display the original total, discount, and final amount.

7. Temperature Converter Menu

Create a function `convert_temperature(value, choice)` for Celsius-to-Fahrenheit and Fahrenheit-to-Celsius conversion. Show a two-option menu, get the temperature and choice from the user, and display the converted result. Use F = C × 9 / 5 + 32 and C = (F – 32) × 5 / 9.

8. ATM Withdrawal

Create a function `withdraw(balance, amount)` that uses nested `if` statements to check that the amount is positive, does not exceed the balance, and is a multiple of 100. If valid, subtract it and display the remaining balance; otherwise, explain why the withdrawal cannot be made. Ask for the starting balance and withdrawal amount.

9. Repeating Arithmetic Menu

Create a function `menu_calculator()` that repeatedly displays a menu: add, subtract, multiply, divide, or exit. Use a `while` loop to keep showing the menu until Exit is selected. For each calculation, ask for two numbers and use conditions to choose the operation. Handle division by zero.

10. Electricity Bill Calculator (Compulsory)

Create a function `electricity_bill(units)` that calculates a bill using these simple rates: first 100 units cost 5 per unit; units above 100 and up to 200 cost 7 per unit; units above 200 cost 10 per unit. Use `if` / `elif` / `else` and arithmetic. Ask the user for units, reject negative input, and print the bill. Example: 150 units costs 100 × 5 + 50 × 7 = 850.

Quick checklist

- Did you define and call a function in every program?
- Did you use input() for the values requested?
- Did you convert numeric input before doing arithmetic?
- Did you handle invalid choices or values where the task asks?
- Did you test at least one normal case and one edge case?